Guion de defensa adaptado — turnos-odontologia (15 min, 4 integrantes)

Guion verificado contra el repo real. Cada dato cita su fuente entre paréntesis.

Repo: https://github.com/jeronimocoronel784-hue/turnos-odontologia

Duración total: 15 min (3 + 4 + 4 + 4). Tiempos por bloque indicados en cada sección.

INTEGRANTE 1 — Etapas 0 y 1: punto de partida y fundación (3 min)

Bloque 1.1 — Etapa 0: entorno y repo (1 min 30)

[MOSTRAR EN PANTALLA: `README.md` **abierto en la tabla de Prerrequisitos + terminal con** `git clone` **del repo]**

"Buenas, arranco yo con la etapa cero, que es el punto de partida. Nuestro proyecto es `turnos-odontologia`: un SaaS multi-tenant para consultorios odontológicos de Argentina — agenda con anti-solapamiento, reserva online 24/7, historia clínica con odontograma, caja y señas con Mercado Pago, obras sociales y recordatorios por WhatsApp. Todo eso está declarado en el `README.md` del repo, no lo estamos inventando ahora.

Para trabajar necesitamos tres cosas, y solo tres: Docker 24 con Compose v2, Python 3.12 y Node 22. Eso está en la tabla de prerrequisitos del README. Aclaro esto porque en el guion anterior se mencionaba Go, y Go no forma parte de este proyecto: el backend es Python con FastAPI y el frontend es React con TypeScript y Vite.

El repo está en GitHub, es público y se clona con un comando. ¿Qué es el harness? Es simplemente el entorno donde corren los agentes de IA: nosotros usamos OpenCode con el Active Stack, que es un conjunto de habilidades y flujos que ordenan el trabajo. Sin harness, cada agente haría lo que quiere; con harness, todos siguen el mismo método: descubrir, documentar, planificar, implementar por cambios chicos y guardar memoria. El primer comando que corrimos fue el del orquestador, que inicializa OpenSpec — y OpenSpec es el sistema que versiona las especificaciones como archivos dentro del propio repo, en la carpeta `openspec/`. O sea: la especificación vive al lado del código, no en un documento perdido en Drive."

Bloque 1.2 — Etapa 1: orquestador y Discovery (1 min 30)

[MOSTRAR EN PANTALLA: `.active-orchestrator-state.json` **— sección** `competitors` **con los 18 sistemas]**

"En la etapa uno ejecutamos el comando del orquestador, que nos preguntó si queríamos hacer Discovery antes de documentar, y dijimos que sí. ¿Por qué? Porque programar sin investigar el mercado es diseñar a ciegas: si no sabemos qué hacen DentalSoft o Dentalink, podemos construir algo que ya existe o, peor, algo que nadie necesita.

La prueba de que el Discovery se hizo de verdad está en el archivo de estado del orquestador: ahí quedaron registrados los 18 competidores relevados y los 5 casos de uso que salieron de esa investigación. Con eso cerramos la etapa uno y habilitamos la etapa dos, que les cuenta mi compañero."

Transición: "Hasta acá: entorno reproducible, repo real y decisión explícita de investigar antes de construir. Ahora viene lo más importante de la metodología: el Discovery con evidencia."

INTEGRANTE 2 — Etapa 2: Discovery con evidencia (4 min)

Bloque 2.1 — Qué es el Discovery y cómo se hizo (2 min)

[MOSTRAR EN PANTALLA: `discovery/discovery.md` **— encabezado con fecha/método + tabla comparativa, filas 1–5]**

"Retomo. ¿Qué es el Discovery? Es la investigación de mercado previa a escribir una sola línea de especificación. Usamos la skill `discovery-research`, que nos obligó a completar un checklist de 11 puntos: problema, usuarios, casos de uso, competidores, funcionalidades necesarias y opcionales, reglas de negocio, integraciones,

restricciones, riesgos y preguntas abiertas. Si falta uno, el discovery está incompleto. Eso está en el Anexo del informe, no es una lista que improvisamos.

Relevamos exactamente 18 sistemas, ni más ni menos: 7 argentinos — DentalSoft, FLAP, ClinIA, Odontoly, Citalo, OdontoSoft Millennium y el legado como DentiOffice —, 2 regionales — Dentalink y Dentidesk de Chile —, 2 españoles — Clinic Cloud y XDentalCloud —, 1 marketplace global que es Doctoralia, 5 estadounidenses y británicos — Tab32, Curve, Dentrix Ascend, CareStack, Dentally y Software of Excellence — y el grupo de sistemas legados argentinos. Cada ficha tiene su URL oficial y fecha de consulta: todo se consultó el primero de octubre de 2026.

Y acá está el punto metodológico que nos diferencia: solo aceptamos fuentes verificables — sitios oficiales, páginas de precios, centros de ayuda, videos oficiales. Todo lo que una empresa afirma pero no demuestra se marca como afirmación comercial, y todo lo que no aparece en público se marca como 'No evidenciado'. Esa honestidad es verificable: el informe tiene decenas de celdas 'No evidenciado', y eso es una virtud, no un defecto. Prefiero decir 'esto no lo pudimos comprobar' antes que copiar marketing como si fuera un hecho."

Bloque 2.2 — Hallazgos y MVP (2 min)

[MOSTRAR EN PANTALLA: matriz de puntuación — fila Dentalink 4.25 + sección C.3 vacíos + D.3 MVP]

"Con ese criterio puntuamos a los 18 sistemas de 0 a 5 en 7 dimensiones con pesos: turnos y automatización pesa 25 por ciento, clínica 20, integraciones locales y WhatsApp 15, administración y cobros 15, experiencia del paciente 10, seguridad y exportación 10, y precio 5. El mejor — Dentalink con 4,25 — pierde justamente en integraciones argentinas: no tiene AFIP, ni Mercado Pago, ni obras sociales nuestras. Nadie supera el 4,30 porque nadie combina clínica profunda con encaje local argentino. Ese es nuestro hueco.

Los hallazgos que justifican el proyecto son cuatro, todos con evidencia. Uno: ningún SaaS argentino relevado muestra facturación AFIP nativa. Dos: solo FLAP muestra Mercado Pago con señas atadas al turno; el resto, nada o un add-on pago. Tres: el WhatsApp automático real — API oficial, plantillas, opt-in, costo por mensaje publicado — no lo tiene documentado casi nadie; Dentidesk lo anuncia como 'próximamente'. Cuatro: solo DentalSoft declara liquidación de obras sociales, sin evidencia de actualización.

De ahí sale el MVP, que está escrito en el informe: agenda multi-profesional y multi-sillón con anti-solapamiento, reserva online sin registro obligatorio, confirmación automática por WhatsApp, ficha clínica con odontograma, caja con señas por Mercado Pago, cálculo de cobertura de obra social contra particular, roles, auditoría mínima y exportación en un clic. Y la regla de corte es explícita: si una funcionalidad no reduce ausentismo, no acelera el cobro o no evita doble carga entre agenda, historia clínica, caja y obra social, va al backlog. Por eso la facturación AFIP completa quedó para la versión 2: en la 1 alcanza con exportación conciliable, y decirlo así es honestidad, no recorte."

Transición: "Ya sabemos qué construir y por qué nadie lo hace bien en Argentina. Mi compañera les cuenta dónde quedó escrito para que ningún agente lo olvide: la base de conocimiento y el roadmap."

INTEGRANTE 3 — Etapas 3, 4 y 5: KB, roadmap y skills (4 min)

Bloque 3.1 — Etapa 3: la base de conocimiento (1 min 30)

[MOSTRAR EN PANTALLA: carpeta `knowledge-base/` con los 12 archivos + `05_reglas_de_negocio.md`, una regla RN-AG]

"La etapa tres convierte el Discovery en la fuente única de verdad: la knowledge base. Usamos la skill `kb-creator` y el resultado son 12 archivos — 11 numerados del 01 al 11 más el README —, no 11 como decía el guion anterior. Los nombro para que se vea que existen: visión y objetivos, descripción general, actores y roles, modelo de datos, reglas de negocio, funcionalidades, flujos principales, arquitectura propuesta, decisiones y supuestos, preguntas abiertas y pagos con Mercado Pago.

¿Por qué importa? Porque sin esto cada agente interpreta el dominio a su manera. Ejemplo concreto: la regla de que no puede haber dos turnos del mismo profesional o sillón en el mismo horario no está en la cabeza de nadie, está escrita en `05_reglas_de_negocio.md` con su código, y el modelo de datos declara 18 entidades donde toda

tabla de negocio lleva `tenant_id`. Otro ejemplo: los 4 roles — dueño, recepción, odontólogo y paciente por token — están en `03_actores_y_roles.md` con su matriz de permisos: recepción nunca edita contenido clínico, odontólogo nunca reabre cierres de caja. Si un agente propone algo que contradice esto, la KB lo desmiente. Además quedaron registradas decisiones con alternativas descartadas — como PWA en vez de app nativa, o monolito FastAPI en vez de microservicios — y 4 preguntas abiertas reales: hosting definitivo, costo de WhatsApp, retención legal y pricing en pesos. Todo trazable."

Bloque 3.2 — Etapa 4: el roadmap (1 min 15)

[MOSTRAR EN PANTALLA: `CHANGES.md` — árbol de dependencias + camino crítico]

"La etapa cuatro ordena la construcción: el archivo `CHANGES.md` define 17 cambios, del C-01 al C-17, en 7 fases — cimientos, acceso, dominio base, flujos asistenciales, dinero, transversal y frontend PWA. El árbol de dependencias dice qué necesita a qué: por ejemplo, la agenda necesita catálogo y pacientes, y Mercado Pago necesita caja. Los gates marcan qué se puede paralelizar: con 3 agentes, catálogo y pacientes van en paralelo, pero la agenda los espera a los dos.

Y lo clave para una defensa: el camino crítico son 9 cambios — C-01, C-02, C-03, C-05, C-06, C-09, C-11, C-12 y C-15. Es el mínimo irreducible: reserva online con seña cobrada por Mercado Pago. Sin el C-15 no hay captura 24/7, sin el C-12 no hay seña. Todo lo demás suma, pero esto es lo que hace que el producto exista. Detalle importante: el ciclo 1 fusionó C-01 y C-02 en un solo change, porque infraestructura sin modelos no verifica nada y modelos sin infraestructura no corren. Eso lo decidió el equipo, no la herramienta."

Bloque 3.3 — Etapa 5: skills y reglas (1 min 15)

[MOSTRAR EN PANTALLA: `AGENTS.md` — tabla de skills + reglas R4, R7, R8, R9, R13]

"La etapa cinco responde: ¿con qué reglas juegan los agentes? Primero, `find-skill` encontró 14 skills reales, y el registry — que sí existe, en `.atl/skill-registry.md` — las asigna por rol: Backend Core con FastAPI y Postgres, Backend Aux con auth, pagos y mensajería, Frontend con React, Clínico con patrones de historia clínica, Infra con Docker, Diagnóstico para WhatsApp y Calidad para reviews. Incluso dice qué skill usa cada change: por ejemplo, pagos con Mercado Pago reusa el patrón de Stripe y de webhooks como plantilla.

Después, `agent-instruction` escribió el `AGENTS.md` con 13 reglas duras, y cito las que más pesan. R4: el `tenant_id` del token JWT manda, nunca se confía en el que manda el cliente — esto evita que un consultorio vea datos de otro. R7: nunca borrado físico en historia clínica, turnos o caja — solo borrado lógico más auditoría. R8: los montos se calculan en el backend y los webhooks son idempotentes — un pago duplicado de Mercado Pago no se cobra dos veces. R9: nunca datos de pacientes en logs ni errores visibles. Y R13: nunca datos reales de pacientes en el repo, solo fixtures sintéticos. Violar cualquiera de estas bloquea el change. Así los agentes tienen libertad dentro de un corral seguro."

Transición: "Ya tenemos qué hacer, en qué orden y con qué reglas. Falta lo que todo tribunal pregunta: ¿funciona? Mi compañero muestra el ciclo real que ejecutamos."

INTEGRANTE 4 — Etapas 6 y 7: OPSX, Engram y cierre (4 min)

Bloque 4.1 — Etapa 6: el ciclo OPSX real (2 min 15)

[MOSTRAR EN PANTALLA: `openspec/changes/archive/2026-10-03-c-01-foundation-core-models/` + `tasks.md` con 10/14 + terminal con `pytest` 22 passed]

"La etapa seis es OPSX, nuestro ciclo de implementación, y es donde corrijo el error más importante del guion anterior: OPSX no tiene cinco etapas. Son cuatro pasos — explorar, proponer, aplicar y archivar — y la verificación vive dentro del aplicar, no es una etapa separada. Decirlo de otra forma sería inventar un proceso que no usamos.

Les cuento el ciclo 1, que es real y está archivado con fecha 3 de octubre de 2026. En explorar decidimos fusionar C-01 y C-02. En proponer generamos los 4 artefactos: proposal con el porqué, design con 7 decisiones — por ejemplo, aislamiento multi-tenant por filtro en repositorio sin RLS en versión 1, documentado y reversible —, 2

specs y lista de tareas. Las specs tienen 18 escenarios en formato Dado-Cuando-Entonces: 7 de infraestructura y 11 de modelos. Ejemplo: dado que en el tenant A existe un email, cuando se crea el mismo email en el tenant B, entonces tiene éxito, porque la unicidad es por tenant. Eso es una regla de negocio convertida en contrato testeable.

En aplicar implementamos 10 de 14 tareas con TDD estricto: FastAPI con health check, modelos Tenant, Usuario y Auditoría, repositorio base con filtro automático por tenant, migración 001, PWA base, Compose y CI. Las 4 restantes quedaron pendientes solo porque no había Docker en la máquina — están identificadas una por una. Y la verificación dio 22 tests pasados más 2 salteados por diseño, porque los de permisos de auditoría exigen el rol restringido de Postgres; más 5 tests nuevos que agregamos para cubrir escenarios sin cobertura. Lint, tipos y build en verde. En archivar, las specs se sincronizaron a `openspec/specs/` y el change quedó cerrado con copia de entrega en `docs/changes/`. Solo se archiva lo aprobado: lo pendiente quedó escrito, no escondido."

Bloque 4.2 — Etapa 7: Engram y conclusión (1 min 45)

[MOSTRAR EN PANTALLA: una memoria Engram del ciclo, ej. Verify 22+2, o `engram` stats: 45 observaciones]

"La etapa siete es Engram, la memoria persistente del proyecto. ¿Por qué existe? Porque los agentes olvidan todo entre sesiones. Engram guarda decisiones, convenciones y descubrimientos con su contexto: qué se hizo, por qué, dónde y qué se aprendió. Ejemplo recuperable: del ciclo 1 quedó guardado que la suite dio 22 pasados más 2 salteados, que los salteos son por diseño y que 4 tareas solo cierran con Docker o en CI. Si mañana otro agente retoma, no redescubre nada: lo lee. Hoy hay 45 observaciones guardadas, incluyendo cada fase del ciclo y el resumen de sesión.

Cierro con los 6 aprendizajes. Uno: investigar con evidencia evita construir lo equivocado — el 'No evidenciado' nos protegió del marketing. Dos: escribir las reglas antes que el código hace que los agentes se autocorrijan. Tres: un roadmap con dependencias y camino crítico convierte un proyecto grande en sesiones de medio día. Cuatro: las reglas duras — tenant del token, sin borrado físico, montos en backend — son seguridad diseñada, no parches. Cinco: especificar con escenarios Dado-Cuando-Entonces hace que testear sea verificar, no improvisar. Y seis: lo que no queda guardado en Engram se pierde; lo que queda, se hereda.

En 15 minutos les mostramos un proyecto organizado, trazable y verificable con IA: cada afirmación que hicimos está en un archivo del repo. Gracias."

Anexo — Qué se corrigió vs guion original

#	Dato original	Dato corregido	Fuente
1	Prerrequisitos incluían Go	Sin Go: Docker 24/Compose v2, Python 3.12, Node 22	`README.md` §Prerrequisitos
2	KB de "11 documentos"	12 archivos (01–11 + README)	`knowledge-base/` (listado real)
3	Discovery: "+15 sistemas" genérico	Exactamente 18 sistemas, relevados 2026-10-01, con URLs y fechas por ficha	`discovery/discovery.md` §A + `docs/discovery/informe-discovery.md`
4	Competidores con nombres aproximados	Nombres reales: DentalSoft, Dentalink, Dentidesk, Clinic Cloud, FLAP, ClinIA, OdontoSoft Millennium, Doctoralia, Odontoly, Citalo, XDentalCloud, Tab32, Curve, Dentrix Ascend, CareStack, Dentally, SOE, legado AR	`discovery/discovery.md` tabla A
5	Hallazgos sin método ("9 aspectos")	Método explícito: solo fuentes verificables; "No evidenciado" vs "afirmación comercial"; matriz 0–5 con 7 pesos; top Dentalink 4.25; 5 demos + 3 refs UX; MVP D.3	`discovery/discovery.md` §§B–D
6	Roadmap sin números precisos	17 changes C-01–C-17, 7 fases, camino crítico de 9 (…C-11→C-12→C-15); ciclo 1 fusionó C-01+C-02	`CHANGES.md`

#	Dato original	Dato corregido	Fuente
7	Skills sin detalle ("registry si existe")	14 skills reales en `.atl/skill-registry.md` (sí existe, generado), 7 roles de agente, tabla Change→Skills; 13 reglas R1–R13 en `AGENTS.md`	`.atl/skill-registry.md`, `AGENTS.md`
8	OPSX en "cinco etapas" con Verify separado	4 pasos: explore→propose→apply→archive; Verify vive dentro de apply	`openspec/changes/archive/2026-10-03-c-01-foundation-core-models/` + convención OPSX
9	OPSX con "3 escenarios" genéricos	18 escenarios reales (7 foundation + 11 core) en Dado/Cuando/Entonces; 14 tasks, 10/14 aplicadas (4 pendientes Docker: 1.4, 2.4, 4.2, 4.3); 22 passed + 2 skipped por diseño; 5 tests nuevos `test_cobertura_verify.py`; ruff+mypy+tsc+build verdes	`openspec/specs/*/spec.md`, `tasks.md`, `tests/`, memoria Engram #16
10	Engram genérico ("decisiones, reglas")	45 observaciones, 4 sesiones; ciclo 1: #13 explore, #14 propose, #15 apply, #16 verify, #17 archive, #18 summary; topic_keys `opsx/ciclo-1/*`	Engram (`mem_stats`, `mem_search`)
11	Reglas citadas sin código	R1–R13 con ejemplos: R4 tenant del JWT, R7 soft delete, R8 montos en backend + idempotencia, R9 sin PHI en logs, R13 fixtures sintéticos	`AGENTS.md` §Reglas Duras
12	Sin fechas/URLs concretas	Repo https://github.com/jeronimocoronel784-hue/turnos-odontologia; change archivado 2026-10-03; PO-01–PO-04 abiertas; `reflexion.md` aún plantilla por completar	`README.md`, `CHANGES.md`, `knowledge-base/10_preguntas_abiertas.md`, `reflexion.md`